Project Proposal

Toxic Prompt Filter for Urdu and Roman-Urdu LLM Inputs

Category: Category 1 – Development of an NLP Application

Overview

This project aims to develop a lightweight safety layer that filters toxic or offensive text in Urdu and Roman-Urdu before it is sent to a Large Language Model (LLM). The purpose is to ensure that user inputs containing hate speech, abuse, or profanity are either blocked or sanitized before being processed. This type of filtering is vital for safe AI deployment in multilingual and low-resource language contexts such as Urdu.

Motivation

While English-language models often include moderation layers, Urdu and Roman-Urdu inputs lack robust safety mechanisms. As a result, conversational systems may inadvertently respond to harmful content or reproduce toxic behavior. The proposed system provides a localized, lightweight solution that promotes ethical and safe use of LLMs for Urdu speakers. By leveraging existing multilingual models and minimal preprocessing, this project demonstrates how AI safety techniques can be adapted for underrepresented languages with low computational cost.

Literature Review

Research on toxicity detection in low-resource languages has grown steadily in recent years, yet Urdu and its romanized form (Roman-Urdu) remain comparatively underexplored. Early studies such as the RUHSOLD corpus introduced around ten thousand annotated Roman-Urdu tweets for hate-speech and offensive-language detection. This dataset demonstrated that supervised learning, even with limited resources, could effectively capture toxic cues in noisy and code-mixed text. Later, the Roman-Urdu Toxic (RUT) Corpus expanded this work by compiling over seventy thousand social-media comments labeled for toxicity, providing stronger coverage of informal spellings and online slang. Similar work on Urdu-script YouTube comments highlighted the impact of domain and script variation on model accuracy. Together, these studies underline the need for models that can handle both scripts and the messy, mixed-language nature of real online communication.

Advances in multilingual transformers—particularly XLM-RoBERTa and multilingual BERT—have significantly improved cross-lingual toxicity classification. These models, trained on hundreds of languages, outperform older monolingual baselines on low-resource languages like Urdu. However, researchers note that Roman-Urdu's lack of standardized spelling often reduces classifier performance. Recent transliteration studies suggest that mapping Roman-Urdu into standardized Urdu script can unify input forms and improve downstream toxicity detection.

Beyond detection, work on prompt moderation and detoxification explores how to sanitize text before generation. Benchmark efforts such as *RealToxicityPrompts* revealed that even harmless inputs can lead large language models to produce toxic completions, prompting development of decoding-control and paraphrasing methods like DExperts or GPT-Detox. These rewriting strategies balance safety and meaning preservation but require careful evaluation to avoid over-filtering.

Finally, multilingual safety remains an open challenge. Studies show that guardrails trained on English are easily bypassed through non-English prompts, underscoring the importance of language-specific filtering and preprocessing. Collectively, this literature supports an Urdu-focused pipeline combining multilingual transformers, transliteration, and prompt rewriting to ensure safe, culturally aware LLM interactions.

Proposed Pipeline

The proposed system follows a modular end-to-end pipeline designed for low-resource deployment and minimal computational overhead. The complete workflow from user input to final model output is described below:

1. User Input and Preprocessing:

The system receives a text prompt in either Urdu script or Roman-Urdu. Basic preprocessing steps are applied, including whitespace trimming, punctuation normalization, and Unicode standardization. The script type is then identified using Unicode range detection to determine whether the text is in Urdu (Perso-Arabic script) or Roman-Urdu (Latin script).

2. Toxicity Classification:

The preprocessed input is passed to a pretrained multilingual toxicity detection model such as 'unitary/toxic-bert' or 'cardiffnlp/twitter-xlm-roberta-base-sentiment', accessed through the Hugging Face Transformers library. The model predicts a toxicity score and label (toxic or safe). No additional training is performed to maintain lightweight implementation.

3. Toxic Text Sanitization:

If the classifier identifies the text as toxic, a rule-based sanitizer is invoked. This module applies regular expression (regex) rules to remove or mask offensive terms using a curated list of common Urdu and Roman-Urdu profanities. For instance, words such as 'bewakoof' or 'badtameez' are replaced with neutral placeholders (e.g., [insult]) while preserving the original sentence structure. The sanitized text is then rechecked by the toxicity classifier to verify that all harmful content has been removed.

4. Blocking Logic:

If the sanitized text remains toxic after re-evaluation, the system blocks the input and returns a polite refusal message in Urdu (e.g., "معذرت — اس میں درخواست میں مدد نہیں کر سکتا"). This prevents the downstream LLM from being exposed to harmful or unsafe content.

5. Safe Prompt Forwarding to LLM:

Clean or verified safe prompts are then forwarded to an external Large Language Model (LLM) such as Google Gemini or the Hugging Face Inference API. The LLM is instructed to produce a response in Urdu through an explicit language directive in the system prompt. The communication with the LLM is managed securely via API calls.

6. Output Verification (Optional):
The generated Urdu response from the LLM is optionally re-evaluated using the same toxicity classifier to ensure that no offensive language is present in the final output. If toxicity is detected, the system either regenerates the response with stricter constraints or replaces it with a safe default message.

7. Response Delivery:
The final, verified safe response is displayed to the user. Throughout the process, minimal metadata (e.g., confidence scores, decision outcomes) may be logged for evaluation purposes while maintaining privacy by not storing raw toxic text.

Summary of Pipeline Flow:
User Input → Preprocessing & Script Detection → Toxicity Classification → Rule-Based Sanitization → Re-classification → Safe/Blocked Decision → LLM Response Generation → Optional Output Check → Final Response.

Datasets

To build and train the toxicity classifier (and support the rewriting module), we will leverage **three** key datasets that cover both Urdu scripts and **Roman Urdu**:

1. Roman Urdu Toxic Comments (RUT) Corpus: A large-scale corpus of Romanized Urdu comments labeled for toxicity. The RUT dataset contains over 72,000 comments collected from social media platforms, each annotated as toxic or non-toxic**.** https://huggingface.co/datasets/hafiz-hassaan-saeed/Roman-Urdu-Toxic-Corpus
2. RUHSOLD (Roman Urdu Hate Speech and Offensive Language Dataset): A focused hate/offensive speech dataset of Roman Urdu text (mainly tweets). RUHSOLD consists of about 10,012 Roman Urdu tweets**.** https://github.com/haroonshakeel/roman_urdu_hate_speech/blob/main/Hate_Words_RomanUrdu_Lower.txt
3. Urdu YouTube Comments (Offensive Dataset): A smaller but important dataset of toxic comments written in native Urdu script, collected from YouTube. Prior work has gathered roughly 2,000 Urdu comments from YouTube videos and annotated them for abusive language**.** https://github.com/pervezbcs/Urdu-Abusive-Dataset/blob/master/Urdu%20Abusive%20Dataset.zip

These three datasets cover both forms of written Urdu and a variety of platforms (social media, Twitter, YouTube). They are all derived from real-world usage, meaning they include the kind of code-mixed, noisy, and diverse language we expect in user inputs. By training on this data, our toxicity classifier will learn to recognize everything from straightforward profanity to context-dependent hate speech in Urdu. We will allocate a portion of each dataset for validation and testing, ensuring that the classifier generalizes well.

Evaluation Plan

The project's performance and effectiveness will be measured using the following evaluation criteria:

- Classifier Accuracy – Evaluate precision, recall, and F1-score for toxic vs. non-toxic classification on the test subset.
- Rewrite Effectiveness – Manually inspect a small set of cleaned examples to confirm that offensive words are removed or replaced appropriately.
- Response Safety – Verify that the final LLM-generated Urdu responses are free from toxic or harmful content.
- System Efficiency – Measure processing time per input to ensure that the safety filter introduces minimal latency.

Expected Outcome

The final deliverable will be a functional prototype that demonstrates the ability to detect and sanitize toxic text in Urdu and Roman-Urdu. The system will block or rewrite harmful inputs before forwarding them to an LLM, thereby reducing the likelihood of unsafe or biased outputs. This project will showcase a practical approach to content moderation for low-resource languages while remaining lightweight and feasible within limited computational and time resources.